

초파리 유충 기반 냄새원 탐색 강화학습

Reinforcement Learning Agent for Olfactory Source Localization Inspired by
Drosophila Larvae Chemotaxis

바이오메디컬공학과

2021070855 인현서

Link: <https://github.com/InHyunseo/Brain-inspired-OSL>

1. 베이스라인 모델 (학습 없음)

학습 없이 동작하는 비례 제어 컨트롤러로, 문제의 난이도를 가늠하고 학습 모델이 넘어야 할 기준선을 설정한다.

- 감각 입력: 양측 센서, 센서 간격 0.15 mm, 좌우 농도 c_L , c_R

제어식:

$$\begin{aligned}\omega_{\text{body}} &= \tanh(80 * G * \text{delta}(R, L)) \\ \text{delta}(R, L) &= (c_L - c_R) / (c_L + c_R)\end{aligned}$$

정규화한 좌우 농도 차이에 비례 계인 80 을 곱하고 $\tanh$ 를 적용해 몸통 방향을 직접 조향한다.

- Cast 규칙: 베이스라인에서 cast(능동 감각)는 명시적 규칙이다. 냄새 신호를 잃으면 cast 로 전환한다. 따라서 노이즈가 커질수록 신호 손실이 잦아져 cast 비율이 거의 기계적으로 증가한다.
- 환경: 노이즈 없음 / 정적 노이즈 / 동적 노이즈

구현: src/baselines/chemotaxis.py 의 BilateralChemotaxis. 신호 손실과 cast 전환은 평활화한 농도의 running max(c_{max} , c_{ewma})와 약신호 및 상승 deadband 로 판정한다.

결과: 소스 방향으로 곧장 조향하며, 노이즈가 커질수록 성공률이 낮아지고 도달 스텝이 늘어난다. cast 비율은 가장 어려운 조건에서 크게 증가한다. 학습 없이도 가우시안 2D 탐색을 잘 풀며, 지금까지 시도한 어떤 강화학습 에이전트보다 우수해 기준선 역할을 한다.

2. 강화학습 프레임워크 (에이전트와 환경)

2.1 에이전트 설계 (유충 형태 모사)

- 몸 길이: 3.5 mm
- 좌우 센서 간격: 0.15 mm
- 센서 전방 오프셋: 1.75 mm
- 머리 스캔 호 반지름: 1.75 mm
- 제어 변수: 전진 속도 v , 몸통 회전 body_omega , 머리 회전 head_omega

head_omega 는 능동 감각을 구조적으로 가능하게 하는 축이다.

- 관측 (6 차원): c_{left} , c_{right} , d_{log} , $\text{prev_}v$, prev_body_omega , prev_head_omega
- 행동 (3 차원): v , body_omega , head_omega , 범위 -1 에서 1, $\tanh$ 로 압축된 가우시안 정책

구현: `src/envs/osl_env.py` 의 `OslEnv` 와 `EnvConfig`, `src/envs/geometry.py` 의 양측 센서 위치와 각도 `wrap`, `src/envs/events.py` 의 `classify_event` 로 `run`, `cast`, `turn`, `spin` 진단 플래그를 만든다.

2.2 환경

기본 환경은 가우시안 농도장이며, 노이즈 배치와 $\text{sigma}(0.5, 1.0)$, 장면 인덱스(1, 2)로 난이도를 조절한다. 이 난이도 파라미터화가 이후 커리큘럼 학습으로 이어진다.

구현: `src/envs/odor_field.py` 의 `GaussianOdorField` 와 `bump-field` 교란. 스칼라 α (0 에서 1)가 모든 bump 파라미터를 동시에 조절한다. `visualize_curriculum_field.py` 로 시각화한다.

3. 정책: PPO 와 GRU

3.1 구조

- Actor: 6 차원 관측이 `GRUCell`(입력 6, 은닉 421)을 거쳐 Linear 층에서 3 개 행동 평균을 출력한다. $\log_{\text{std}}$ 는 상태에 무관한 파라미터로 둔다. 순환 정책이며 약 540K 파라미터이다. GRU 가 매 스텝 냄새 정보를 시간적으로 통합하는 점이 커넥툼 정책과의 핵심 차이이다.
- Critic: 별도의 상태 비의존 MLP 로 약 4.7K 파라미터이며 Actor 와 특징을 공유하지 않는다.

구현: src/models/policy.py 의 Policy, CriticMLP, CriticGRU (log_std_init=-0.5, gru_hidden=421), src/models/gru_backbone.py, src/agents/ppo_agent.py 의 PPOTrainer (GAE, 시퀀스 업데이트, persistent rollout state).

3.2 커리큘럼 학습

난이도를 S0 에서 S1, S2 로 높인다. 성공률은 약 0.55 까지 오른 뒤 가장 어려운 S2(sigma 0.6)에서 다시 떨어진다. 이는 현재 모델 용량과 학습량의 한계로, 노이즈가 심해 업데이트가 거의 무작위가 되어 학습이 무너진다. 따라서 이 하락 직전의 체크포인트를 추론에 사용한다.

구현: 커리큘럼 단계는 src/utils/config.py 의 DEFAULT_PHASES_JSON (noise_stage, noise_strength, timesteps 리스트)에 정의하며, train.py 에서 runner.set_noise_stage 로 진행한다.

3.3 핵심 결과

노이즈가 커질수록 성공률은 낮아지고 스텝은 늘어나며, cast(능동 감각) 비율이 뚜렷하게 증가해 가장 어려운 조건에서 10 퍼센트대 후반에 이른다.

- 베이스라인에서 노이즈 증가에 따른 cast 증가는 손으로 만든 규칙(신호 손실 시 cast)의 직접 결과다.
- GRU 에는 그런 규칙이 없고 연속적인 head_omega 만 출력하는데도 같은 방향의 추세가 학습만으로 나타난다.
- 명시적 규칙 모델과 규칙 없는 학습 모델이 같은 경향을 보인다는 점이 이 패턴을 의미 있게 만든다.

3.4 야코비안 고윳값 분석

은닉 동역학의 한 스텝 야코비안 고윳값을 행동별로 분리해 본다. 지배 모드(가장 큰 절댓값 고윳값)를 표시해 Run 구간과 능동 감각 구간의 동역학을 비교한다.

구현: analysis/osl2d/phase3a_jacobian.py

4. 코드 구성 요약

항목	코드
베이스라인 비례 제어	src/baselines/chemotaxis.py
에이전트와 환경 (관측, 행동, 센서, 기하)	src/envs/osl_env.py, geometry.py, events.py

가우시안 농도장과 bump 노이즈, alpha	src/envs/odor_field.py, visualize_curriculum_field.py
GRU actor 와 MLP critic, log_std	src/models/policy.py, gru_backbone.py
커넥톰 actor (message passing)	src/models/connectome.py
PPO 학습, GAE, 커리큘럼	src/agents/ppo_agent.py, train.py, src/utils/config.py
야코비안 고윳값 분석	analysis/osl2d/phase3a_jacobian.py
평가와 최적 에피소드 GIF	eval.py, src/utils/plotter.py